

public class bellaybestia {

// Aventura Conversacional en Java

```
String[] authors = {"Luis", "Guillermo", "Jorge"};
```

```
    Project.version = 1.0;
```

```
// Proyecto Final DAM 1º Año
```



Reinventando un Clásico

```
return "RPG de Consola";
```

Transformamos "La Bella y la Bestia" en una experiencia interactiva basada en texto puro.

- > **Narrativa Ramificada:** Estructuras if/else complejas.
- > **3 Personajes Jugables:** Historia interactiva para cada uno de ellos.
- > **100% Java:** Sin motor gráfico, solo lógica pura.



Selección de Personaje



Bella.java

// La Inteligente

Enfocada en lógica, puzzles y puntos de bondad. Combate dialéctico.



Bestia.java

// El Maldito

Gestión de tiempo (pétalos--).
Dilemas morales y fuerza bruta.



Gaston.java

// El Cazador

Economía, sistema de loot (inventario) y fama en el pueblo.

Camino 1: Lógica de Bella



Camino Bella

El camino de Bella evita la violencia física, premiando la lógica.

- > **Input Parsing:** Adivinanzas de texto exacto (Ding Dong).
- > **Minijuegos:** Adivinar números (Random) con Chip.
- > **Objeto Clave:** Espejo Mágico (variable boolean hasMirror).

Camino 2: Bestia

```
while (petalos > 0)
```

La maldición consume a la Bestia. Un contador global decrece con cada mala decisión.

```
if (ritualComplete && bondad > 50) {  
    becomeHuman();  
} else {  
    badEnding();  
}
```

Decide entre ganar poder oscuro (fácil) o mantener la humanidad (difícil).



Camino 3: Gastón



Gestión de Recursos

Gastón introduce mecánicas de economía y de Loot.

- > **Caza:** RNG para obtener items (Piel de Lobo, Carne).
- > **Mercado:** Convierte items en int gold.
- > **Tienda:** Compra mejoras (Punta Sigilosa) para alterar las probabilidades de éxito en combate.

Sistema de Combate

```
if(ataque == 1){  
    danio = 15;  
    mensajeAtaque = "Argumentas con razón."  
}  
else if(ataque == 2){  
    danio = 10;  
    mensajeAtaque = "Le tiras una enciclopedia."  
}  
else if(ataque == 3){  
    danio = 25;  
    mensajeAtaque = "Le miras a los ojos."  
}  
else{  
    danio = 0;  
    mensajeAtaque = "Te quedas paralizada."  
}
```

Bucle Principal

El combate corre dentro de un do-while hasta que una condición de vida llega a cero.

```
int damage = (int)(Math.random() * 20);  
if (Math.random() < 0.15) {  
    System.out.println("¡Esquivado!" );  
}
```

Implementamos evasión (15%), críticos y daño variable.

Interfaz de Consola

ANSI Escape Codes

Para mejorar la UX en una aplicación de texto, usamos códigos de color.

- > RED para enemigos y errores.
- > GREEN para éxito y vida.
- > BLUE para menús e información.

Se integra arte ASCII para representar el castillo y los personajes.



Historial de Desarrollo

Init Commit

- Estructura básica y bucle principal main.

Feature: Puzzles

- Implementación de lógica para Bella y minijuegos matemáticos.

Refactor: Combat

- Equilibrado de daño y probabilidades (RNG) para Bestia y Gastón.

Release v1.0

- Integración de colores ANSI, arte ASCII y testeo final.



¿Preguntas?

```
while (questions) {  
    answer();  
}  
System.exit(0);
```

// Gracias por su atención

Luis, Guillermo & Jorge